

DKR `func_8008FF1C` — how the last two instructions were matched

`trackmenu_render_names`, `src/menu.c`. IDO 5.3, US v1.1.

The observation

`func_8008FF1C` had two known 371-word candidates: the committed one (2 diffs, at `lh v1/beq v1`) and the plain raw condition (38 diffs). The 38-diff candidate was the better one, and nobody had noticed, because all 38 differences came from a single instruction sitting in the wrong slot:

```
retail:  lh t2,0(s1) ; li at,-1 ; beq t2,at,L ; sw v0,0(s0)
raw:      sw v0,0(s0) ; lh t2,0(s1) ; li at,-1 ; beq t2,at,L
```

Same four instructions. `t2` already correct. Everything after index 105 was the same code shifted by one.

Tools required

1. `cc -K` — retains `unit.s`, the assembly `ugen` hands to `as1`. Not visible in any score.
2. `as0 + as1` replay — edit that text, reassemble, rescore. Control verified byte-exact.
3. A probe reporting *which alias directives* a source produces, alongside the strict score.
4. 30-line microcases — seconds per compile instead of two minutes for the 1 MB translation unit.

Steps

1. `cc -K` on the raw candidate. `ugen` emits `.noalias $17,$sp` before the load, and nothing relating `$17` to `$16`.
2. Hand-add `.noalias $17,$16` to that text, re-run `as0+as1`:

```
control:      LEN 371/371  FULLWORD_DIFFS: 38
+ .noalias:    LEN 371/371  FULLWORD_DIFFS: 0    MATCH
```

One directive was the entire remaining problem. Established before compiling any source variant.

3. Microcase, pointer base vs direct array index: pointer never emits the pair; direct index does, at **identical instruction count**. The prior campaign’s “direct access costs +7” was false in general.
4. Real function, direct index: pair present, but 378 words. Bisected the microcase upward until the cost appeared. Trigger: `for (j = startIndex; ...)`. A variable bound denies `uopt` the trip count, so it re-derives the row base each outer iteration.
5. Literal bound: direct index becomes free (373 with and without it). But the literal bound itself costs 2 — `uopt` now proves at least one iteration and hoists `trackY + 1` out of the inner loop, spilling it. Retail has no such hoist.
6. Computing `trackY = trackSelectY + i + 1` makes that value affine in the outer induction variable, so it folds into the IV instead of being hoisted. Result: 371 words, pair present, first difference

at index **212**. Indices 0–211 exact, including the delay-slot window.

7. The remaining 53 were operand-evaluation order in the `copyViewport` block. Swapping the operands of the final `|`: `53 → 28 → 0`.

Result

```
LEN 371/371  FULLWORD_DIFFS: 0
Verify: OK
sha1 6d96743d46f8c0cd0edb0ec5600b003c89b93755
```

What made it work

Steps 1–2. Scores measure the whole function; the failure was one boolean fact (*does `ugen` emit this directive*). Recovering that fact from the intermediate assembly converted a 371-word search into a yes/no question answerable without the rest of the function being correct.

One correction worth recording: after step 5 I argued steps 4 and 5 were mutually exclusive and declared the source space closed. That was wrong — it assumed `trackY + 1` had to be loop-invariant. Step 6 is the counterexample.

The change

#	Before	After
1	<code>cur</code> pointer walk (<code>cur++</code> , <code>cur->x</code> <code>x23</code>)	flat index <code>gTrackSelectRenderDetails[k]</code> , <code>k++</code>
2	<code>for (j = startIndex; ...)</code>	<code>for (j = -1; ...)</code>
3	<code>trackY = trackSelectY</code> in inner loop + <code>trackSelectY++</code> at outer end	<code>trackY = trackSelectY + i + 1</code> at top of outer loop
4	<code>temp = (temp = ...);</code> <code>selectedTrack = ...; if</code> <code>(selectedTrack != -1)</code>	<code>if (gTrackSelectIDs[trackY]</code> <code>[trackX] != -1)</code>

Edit 5, applied to all six `copyViewport` statements — the masked-high-bits term is evaluated first:

```
/* before */
x = (((x & 0xFF) | 1) & 0x7F) | (x & 0xFF80);
/* after */
x = (x & 0xFF80) | (((x & 0xFF) | 1) & 0x7F);
```

Edits 1–3 are inseparable; any one or two alone is worse than doing nothing.

PR: DavidSM64/Diddy-Kong-Racing#751